

Parallel N-Body Simulation using MPI and Barnes–Hut Approximation

Nicholas Babineaux

April 17, 2026

Contents

1	Introduction	2
2	Methods	2
2.1	All-Pairs Algorithm	2
2.2	Barnes–Hut Algorithm	2
2.3	Time Integration	3
2.4	Parallelization Strategy	3
2.5	Implementation Details	3
3	Experimental Setup	5
3.1	Problem Parameters	5
3.2	Measured Quantities	6
3.3	Expected Performance	6
4	Results	7
4.1	Runtime vs Particle Count	7
4.2	Accuracy vs Particle Count	8
4.3	Representative Numerical Results	8
4.4	Parallel Scaling	10
4.5	3D Simulation Visualization	13
5	Discussion	13
6	Limitations	14
7	Conclusion	14

1 Introduction

The N-body problem is a classical computational physics problem in which a set of particles evolves under pairwise interactions such as gravity. In a direct formulation, the force on each particle is computed by summing contributions from every other particle. This yields a computational complexity of $O(N^2)$, which becomes increasingly expensive as the number of particles grows.

This project implements and compares two approaches:

- a direct all-pairs force calculation with $O(N^2)$ complexity, and
- the Barnes–Hut approximation, which reduces the cost to approximately $O(N \log N)$ by grouping distant particles hierarchically [1].

Both approaches are parallelized using MPI [3] and executed on the Frontera supercomputer. The primary goals of the project are:

- to measure performance as problem size increases,
- to evaluate strong scaling across multiple MPI ranks, and
- to analyze the accuracy/performance tradeoff introduced by the Barnes–Hut approximation parameter θ .

The resulting comparison highlights a central theme in high performance computing: improved asymptotic complexity can reduce runtime significantly at large scales, but only at the cost of additional algorithmic overhead and, in approximate methods, reduced physical fidelity.

2 Methods

2.1 All-Pairs Algorithm

In the direct all-pairs method, the force on particle i is computed by summing contributions from all other particles $j \neq i$:

$$\mathbf{F}_i = \sum_{j \neq i} G \frac{m_i m_j}{\|\mathbf{r}_i - \mathbf{r}_j\|^3} (\mathbf{r}_j - \mathbf{r}_i),$$

where G is the gravitational constant, m_i and m_j are particle masses, and $\mathbf{r}_i$ and $\mathbf{r}_j$ are particle positions. This method is straightforward and highly accurate, but its $O(N^2)$ cost makes it expensive for large systems.

In this project, the all-pairs method serves as the reference solution for accuracy. Because the exact pairwise force calculation preserves equal-and-opposite interactions, total momentum should remain effectively constant up to floating-point roundoff. For that reason, momentum drift is used as a compact diagnostic for accumulated numerical or approximation error.

2.2 Barnes–Hut Algorithm

The Barnes–Hut algorithm reduces computational cost by replacing distant groups of particles with a single aggregate mass at their center of mass. In three dimensions, this is implemented using an octree.

For a tree node of width s at distance d from the target particle, the node is approximated if

$$\frac{s}{d} < \theta,$$

where θ is the opening-angle parameter. Smaller values of θ force more exact traversal and therefore improve accuracy, while larger values increase the amount of approximation and typically improve performance.

This produces an approximate complexity of $O(N \log N)$, though the actual runtime also depends on the cost of tree construction, the particle distribution, and the choice of θ .

2.3 Time Integration

The simulation updates particle velocity and position using a simple explicit timestep update. For each particle, acceleration is computed from either the all-pairs method or the Barnes–Hut traversal, and then the state is advanced:

$$\mathbf{v}^{n+1} = \mathbf{v}^n + \mathbf{a}^n \Delta t, \quad \mathbf{r}^{n+1} = \mathbf{r}^n + \mathbf{v}^{n+1} \Delta t.$$

A small softening term is included in the denominator of the force expression to avoid singular behavior for very small particle separations.

2.4 Parallelization Strategy

MPI is used to distribute work across multiple ranks. The parallel design is intentionally simple and emphasizes correctness, reproducibility, and scalability analysis.

The particle array is divided into contiguous chunks, and each rank owns a subset of particles. At each timestep:

1. all particle data is gathered so that each rank has a consistent global view of the system,
2. each rank computes forces only for its local subset, and
3. each rank updates only its local particles.

In all-pairs mode, each rank computes exact forces on its local particles using the full replicated particle set. In Barnes–Hut mode, each rank constructs the same octree from the gathered global particle data and traverses that tree for its local subset.

This design has two important consequences:

- It simplifies correctness and debugging because every rank sees the same global particle state.
- It increases memory usage and communication cost because the entire particle set is replicated across ranks at every timestep.

This tradeoff was acceptable for the project because it allowed the study to focus on algorithmic behavior and strong-scaling trends without introducing the additional complexity of distributed tree construction.

2.5 Implementation Details

The code is written in C++ and built with CMake. MPI is used for communication, and the program supports multiple runtime configurations:

- `mode=allpairs`

- mode=barneshut
- configurable particle count N
- configurable timestep count
- configurable timestep size Δt
- configurable Barnes–Hut opening angle θ

The following code snippet illustrates the use of MPI collective communication to reconstruct the global particle state from local particle subsets.

Listing 1: Global particle reconstruction using MPI.Allgatherv.

```
std::vector<Particle> gather_global_particles(
    const std::vector<Particle>& local_particles,
    int global_particle_count,
    const std::vector<int>& packed_counts,
    const std::vector<int>& packed_offsets
) {
    std::vector<double> packed_local = pack_particles(local_particles);
    std::vector<double> packed_global(global_particle_count *
        PARTICLE_DOUBLES);

    MPI_Allgatherv(
        packed_local.data(),
        static_cast<int>(packed_local.size()),
        MPI_DOUBLE,
        packed_global.data(),
        packed_counts.data(),
        packed_offsets.data(),
        MPI_DOUBLE,
        MPI_COMM_WORLD
    );

    return unpack_particles(packed_global);
}
```

The Barnes–Hut local update routine builds an octree from the replicated global data, then computes accelerations only for the rank’s owned particles.

Listing 2: Local Barnes–Hut update using a global octree.

```
void step_particles_barnes_hut_from_global(
    std::vector<Particle>& local_particles,
    const std::vector<Particle>& global_particles,
    int global_start_index,
    double dt,
    double theta
) {
    Cube root_region = make_bounding_cube(global_particles);
    OctreeNode root(root_region);

    for (int i = 0; i < static_cast<int>(global_particles.size()); ++i) {
        root.insert(i, global_particles);
    }
}
```

```

}

root.compute_mass_properties(global_particles);

std::vector<Vec3> accelerations(local_particles.size(), Vec3(0.0, 0.0,
0.0));

for (std::size_t i = 0; i < local_particles.size(); ++i) {
    const int global_index = global_start_index + static_cast<int>(i);

    accelerations[i] = root.compute_acceleration_on_particle(
        global_index, global_particles, theta, SOFTENING, G
    );
}

for (std::size_t i = 0; i < local_particles.size(); ++i) {
    local_particles[i].acceleration = accelerations[i];
    local_particles[i].velocity += local_particles[i].acceleration *
        dt;
    local_particles[i].position += local_particles[i].velocity * dt;
}
}

```

Finally, the program reports the maximum elapsed time across ranks, which is the appropriate parallel runtime metric.

Listing 3: Use of the maximum rank time for benchmark reporting.

```

const double local_elapsed = end_time - start_time;
double max_elapsed = 0.0;

MPI_Reduce(&local_elapsed, &max_elapsed, 1, MPI_DOUBLE, MPI_MAX, 0,
    MPI_COMM_WORLD);

```

3 Experimental Setup

All experiments were run on the Frontera supercomputer using MPI.

3.1 Problem Parameters

The following particle counts were used for the size-sweep study:

$$N = 1000, 5000, 10000.$$

The timestep count and timestep size were fixed at:

$$\text{steps} = 10, \quad \Delta t = 0.001.$$

For Barnes–Hut, two opening-angle values were tested:

$$\theta = 0.5, 0.2.$$

For the parallel scaling study, the number of MPI ranks was varied as:

$$p = 1, 2, 4, 8.$$

For larger particle counts, the initial conditions were generated from a fixed random seed so that runs were reproducible across repeated executions and across different algorithmic modes.

3.2 Measured Quantities

The following quantities were recorded:

- elapsed wall-clock time,
- average time per step,
- mass drift,
- momentum drift magnitude,
- speedup, and
- parallel efficiency.

Mass drift is reported primarily as a sanity check; all tested configurations preserved total mass exactly. Momentum drift is used as a practical measure of physical error. The direct all-pairs method serves as the accuracy reference.

3.3 Expected Performance

Before running experiments, the expected trends were:

- The all-pairs method should scale quadratically with N .
- Barnes–Hut should become more efficient than all-pairs once N is large enough for the lower asymptotic complexity to dominate tree-construction overhead.
- Decreasing θ should improve accuracy but increase runtime.
- Both methods should exhibit strong scaling, though Barnes–Hut may scale somewhat less ideally because of tree-related overhead and irregular work per particle. A weak-scaling study would also be informative, but was beyond the scope of the present work.

4 Results

4.1 Runtime vs Particle Count

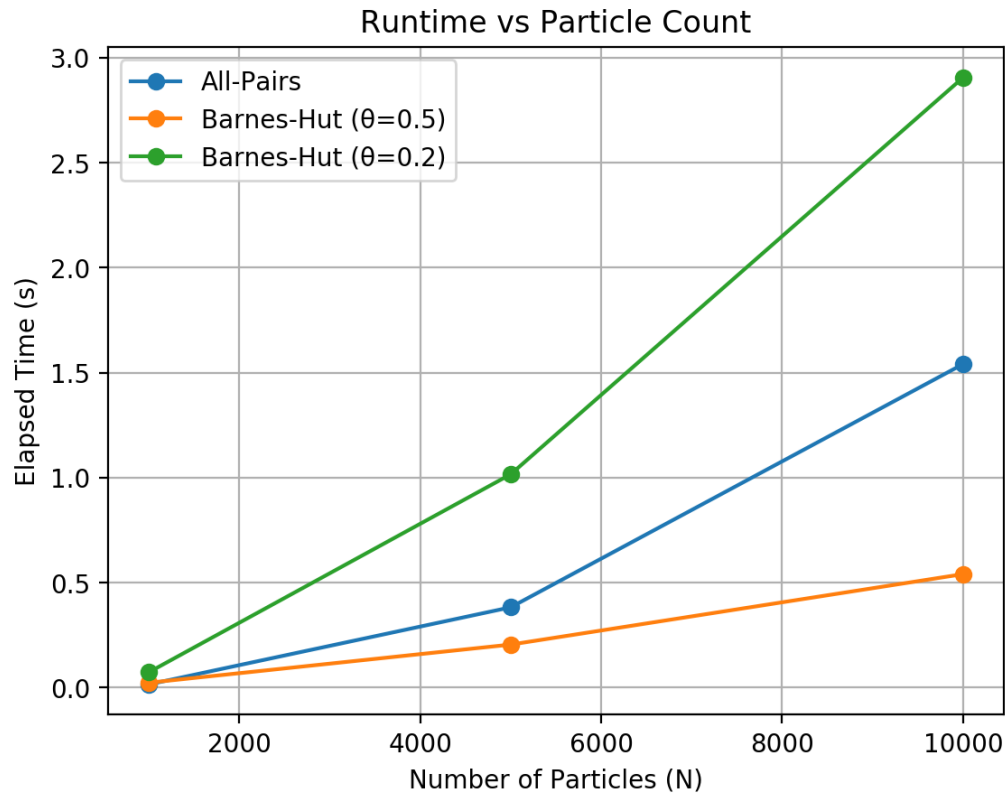


Figure 1: Runtime as a function of particle count.

Figure 1 shows the expected crossover behavior between the two algorithms. At $N = 1000$, the all-pairs method is still faster because the problem is too small for Barnes-Hut’s asymptotic advantage to overcome its tree-construction and traversal overhead. By $N = 5000$, Barnes-Hut with $\theta = 0.5$ becomes faster than all-pairs, and by $N = 10000$ the difference is substantial.

The higher-accuracy Barnes-Hut configuration with $\theta = 0.2$ is slower than both the direct method and the more aggressive Barnes-Hut setting in this implementation. This indicates that reducing approximation error can eliminate the performance advantage of the hierarchical method at these scales.

4.2 Accuracy vs Particle Count

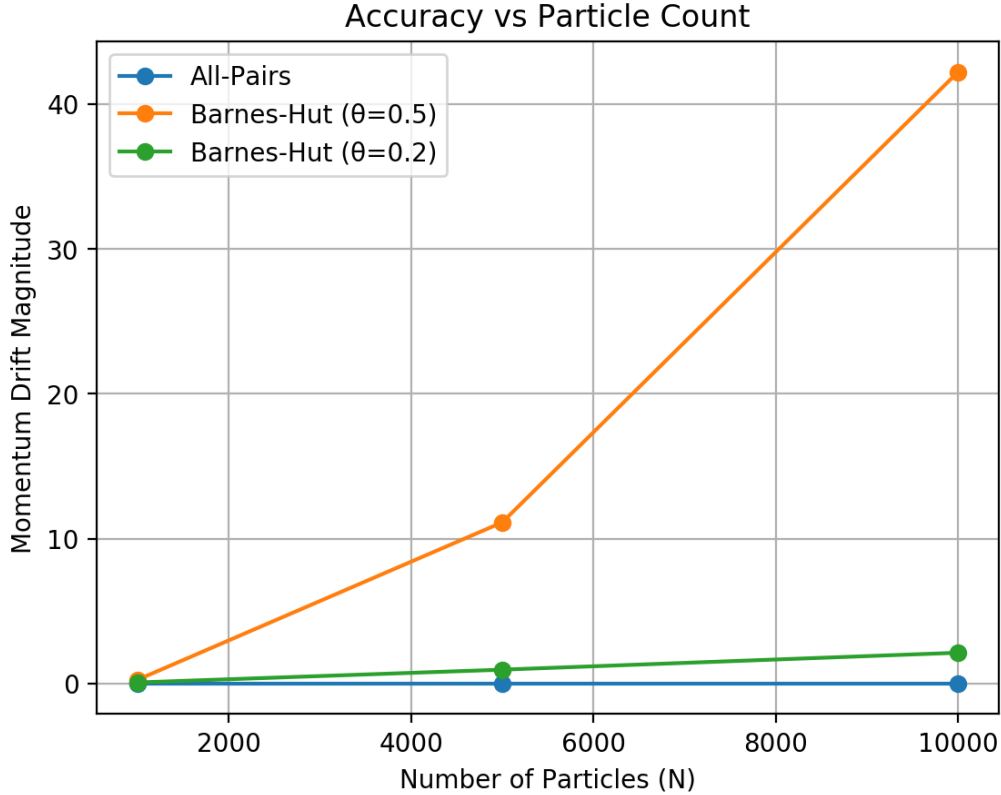


Figure 2: Momentum drift as a function of particle count.

Figure 2 shows the momentum drift magnitude for each method. The all-pairs method remains effectively at machine precision across all tested values of N , which confirms that the reference implementation is behaving as expected.

Barnes-Hut introduces visible momentum drift because the approximate force computation does not preserve exact equal-and-opposite pair interactions. This effect becomes stronger as N increases and as θ increases. In particular, $\theta = 0.5$ produces significantly larger drift than $\theta = 0.2$. The plot therefore captures the essential performance/accuracy tradeoff in the Barnes-Hut method.

Because the all-pairs drift is several orders of magnitude smaller than the Barnes-Hut drift, it appears visually pinned to zero on this scale.

4.3 Representative Numerical Results

Table 1 summarizes representative runtime and drift values for the size sweep.

Table 1: Representative size-sweep results for 4 MPI ranks.

Method	N	Elapsed Time (s)	Momentum Drift Magnitude
All-Pairs	1000	0.015646	6.43504×10^{-11}
Barnes-Hut ($\theta = 0.5$)	1000	0.023734	0.263439
Barnes-Hut ($\theta = 0.2$)	1000	0.073444	0.0727827
All-Pairs	5000	0.382920	8.32074×10^{-11}
Barnes-Hut ($\theta = 0.5$)	5000	0.205427	11.1264
Barnes-Hut ($\theta = 0.2$)	5000	1.016350	0.955454
All-Pairs	10000	1.541220	4.00174×10^{-10}
Barnes-Hut ($\theta = 0.5$)	10000	0.540942	42.2081
Barnes-Hut ($\theta = 0.2$)	10000	2.906000	2.12410

Table 1 makes the crossover and tradeoff clear: the all-pairs method is the most accurate in every case, Barnes-Hut with $\theta = 0.5$ is the fastest at larger N , and Barnes-Hut with $\theta = 0.2$ offers improved accuracy at a substantial performance cost.

4.4 Parallel Scaling

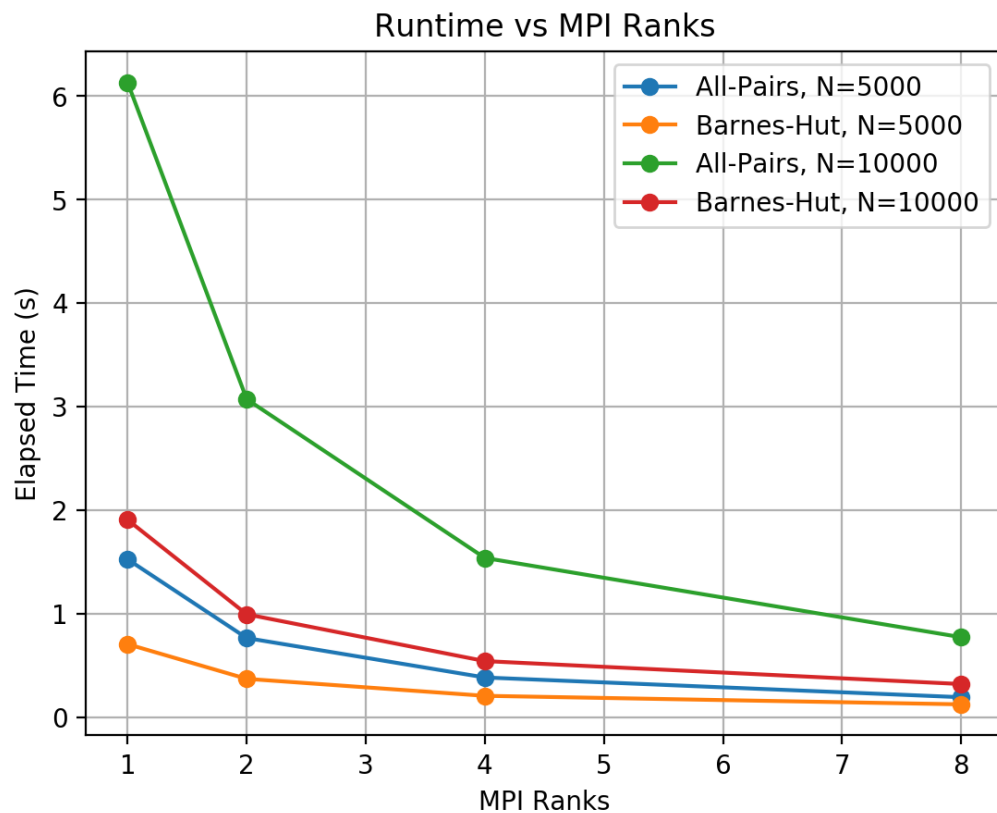


Figure 3: Runtime vs MPI ranks.

Figure 3 shows that runtime decreases as the number of ranks increases for both methods and for both tested values of N . Because the particle count is held fixed while the number of MPI ranks varies, these experiments constitute a strong-scaling study. The all-pairs method displays especially regular behavior because each particle update requires a nearly uniform amount of work.

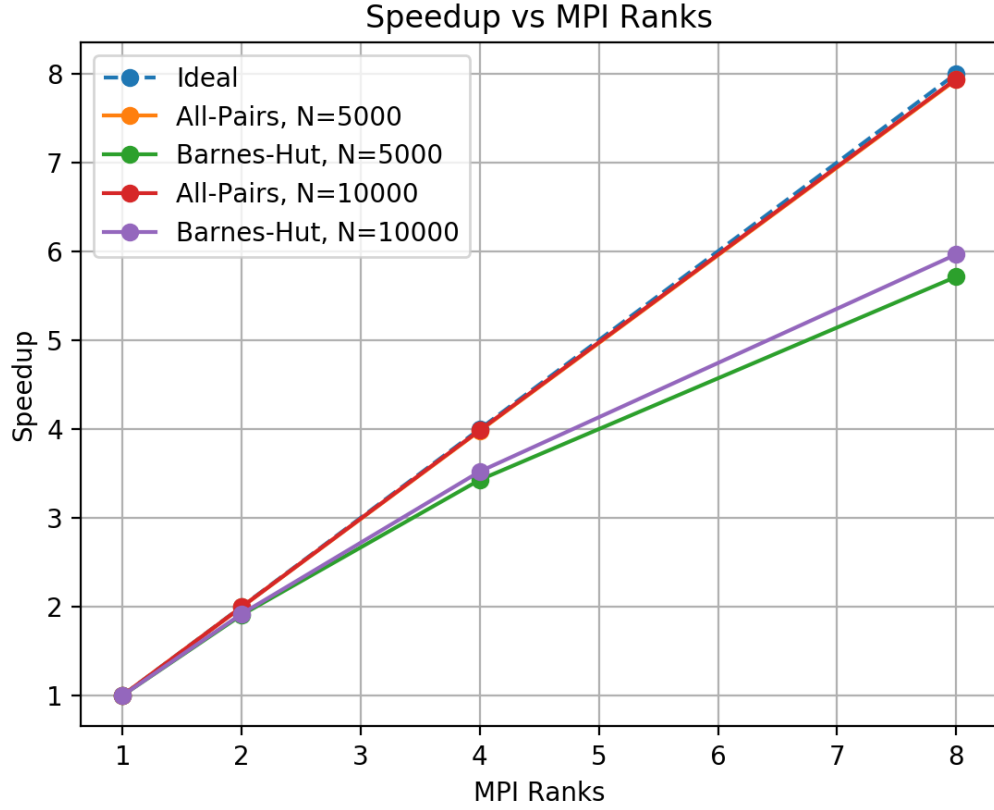


Figure 4: Speedup vs MPI ranks.

Figure 4 compares measured speedup against ideal linear speedup. The all-pairs method approaches ideal scaling very closely, while Barnes–Hut scales somewhat less efficiently. This is expected: although the Barnes–Hut algorithm reduces total computational work, it introduces irregular tree traversal patterns and additional overhead in tree construction, consistent with the general limitations on parallel speedup described by Amdahl’s Law [2].

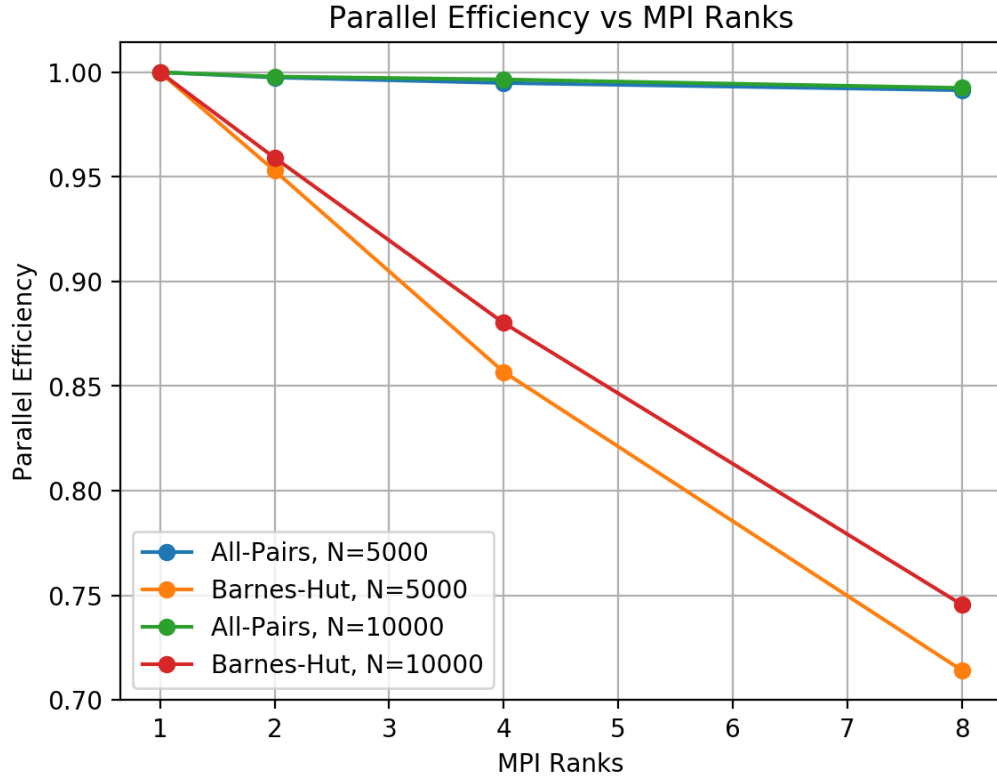


Figure 5: Parallel efficiency vs MPI ranks.

Figure 5 shows parallel efficiency. The all-pairs method remains close to ideal efficiency across the tested rank counts, while Barnes–Hut loses some efficiency at larger rank counts. This reduction is consistent with increased communication overhead, replicated data movement, and less regular per-particle work in the tree-based method.

4.5 3D Simulation Visualization

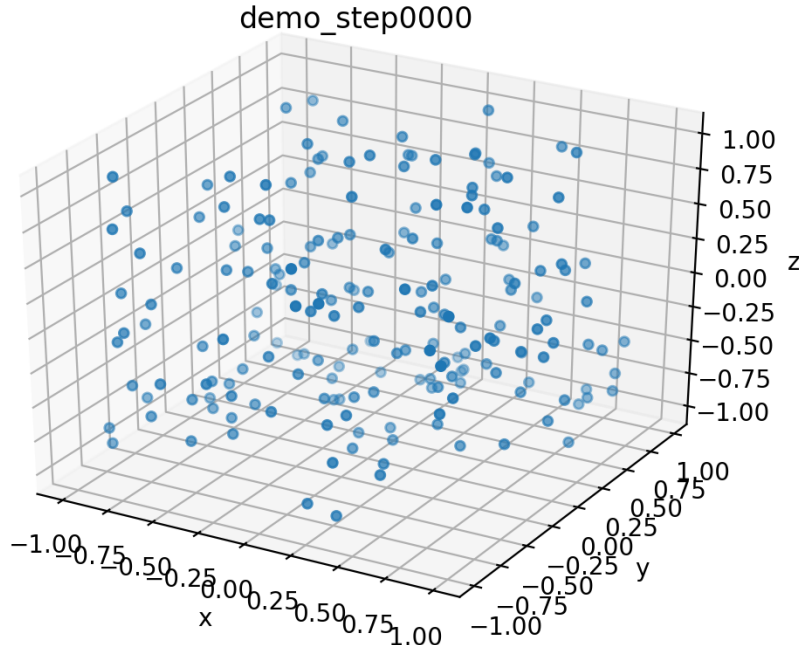


Figure 6: Initial 3D particle distribution used in the simulation.

Figure 6 is included as an illustrative visualization of the three-dimensional simulation domain rather than as a quantitative benchmark result. Only the initial particle configuration is shown for clarity; later timesteps exhibit clustering and outlier behavior that are less suitable for a compact report figure.

5 Discussion

The results are consistent with the theoretical expectations described earlier. The direct all-pairs method is the most accurate configuration tested, with momentum drift remaining at machine precision in every reported experiment. This is expected because each particle interaction is computed explicitly and pairwise.

Barnes–Hut exhibits the opposite pattern: it becomes increasingly attractive in runtime as N grows, but only by accepting force approximation error. For the current implementation, the crossover point between all-pairs and Barnes–Hut with $\theta = 0.5$ occurs between $N = 1000$ and $N = 5000$. By $N = 10000$, the Barnes–Hut method is significantly faster than all-pairs, but the observed momentum drift is also much larger.

The opening angle θ plays the central role in controlling this tradeoff. A smaller value such as $\theta = 0.2$ forces the traversal to behave more like the direct method and therefore greatly reduces momentum drift. However, the resulting runtime cost is high enough in this implementation that the $\theta = 0.2$ configuration is slower than all-pairs for the tested values of N . In contrast, $\theta = 0.5$ yields a clear speed advantage at larger N , but with visibly degraded conservation properties.

The parallel scaling results are also meaningful. The all-pairs method scales almost ideally because the work per local particle is regular and the computation is dominated by force accumulation.

Barnes–Hut still scales well, but somewhat less ideally. This is consistent with the algorithm’s irregular work distribution, tree-construction overhead, and the replicated-data communication strategy used here.

Taken together, the results support the following interpretation:

- all-pairs is the accuracy baseline,
- Barnes–Hut with moderate θ is the practical high-performance mode for larger systems, and
- smaller θ values improve physical fidelity but can eliminate the performance advantage of the hierarchical method.

6 Limitations

Several limitations remain in the current implementation.

First, the Barnes–Hut tree is constructed independently on every rank from replicated particle data. This simplifies the implementation substantially, but it also increases both communication cost and memory usage.

Second, all particle data is gathered globally at each timestep. This design makes correctness and debugging easier, but it is not the most scalable possible approach. A more advanced implementation would use a distributed tree or a more sophisticated domain decomposition.

Third, no dynamic load balancing is implemented. This is more important for Barnes–Hut than for all-pairs, since the amount of tree traversal work per particle can vary.

Finally, the explicit timestep update used here is simple and effective for the purposes of this project, but long runs or more physically demanding initial conditions would likely require a more robust integration scheme.

A further limitation of the current study is that it focuses on strong scaling only. The number of MPI ranks was varied while keeping the particle count fixed, which is useful for measuring how well the implementation accelerates a fixed problem. However, large HPC applications are often also evaluated using weak scaling, in which the problem size grows with the number of ranks so that the work per rank remains approximately constant. A weak-scaling study would provide additional insight into how the replicated-data communication strategy used here behaves as the overall problem size and process count increase together. This was not included in the present work, but it is a natural extension for future evaluation.

7 Conclusion

This project implemented and analyzed a parallel N-body simulation using MPI on the Frontera supercomputer. Two methods were studied: a direct all-pairs formulation and the Barnes–Hut approximation.

The all-pairs method provided the accuracy baseline and exhibited excellent parallel scaling, but its $O(N^2)$ complexity caused runtime to grow rapidly with particle count. The Barnes–Hut method reduced runtime significantly for larger systems, especially at $\theta = 0.5$, but introduced a measurable loss of momentum conservation. Smaller values of θ reduced this error, but at substantial additional cost.

Overall, the project demonstrates several important HPC ideas:

- algorithmic complexity strongly affects practical performance,

- approximation can be traded for speed,
- communication strategy influences scalability, and
- parallel performance must be evaluated together with numerical behavior.

The resulting implementation satisfies the project goals while also identifying clear directions for future improvement, including distributed tree construction, better load balancing, and improved time integration.

References

- [1] J. Barnes and P. Hut, *A Hierarchical $O(N \log N)$ Force-Calculation Algorithm*, Nature, Vol. 324, 1986, pp. 446–449.
- [2] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities*, AFIPS Conference Proceedings, Vol. 30, 1967, pp. 483–485.
- [3] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, Version 4.1, 2023.